

LOJA VELOZ

Relatório Técnico

Cloud DevOps: Orchestrating Containers and Microservices

Formato	Repositório GitHub + Relatório Técnico
Serviços	API Gateway, Pedidos, Pagamentos, Estoque
Tecnologias	Docker, Kubernetes, GitHub Actions

1. Introdução

Este documento apresenta a implementação prática de uma plataforma de e-commerce utilizando arquitetura de microsserviços, containerização com Docker, orquestração em Kubernetes e pipeline CI/CD automatizado. O projeto foi desenvolvido para a Loja Veloz, uma empresa de e-commerce que necessitava modernizar sua infraestrutura para suportar picos de tráfego e permitir deploys contínuos sem indisponibilidade.

A solução implementa 4 microsserviços independentes: API Gateway (responsável por roteamento, autenticação e rate limiting), Pedidos (gestão do ciclo de vida de pedidos), Pagamentos (processamento de transações) e Estoque (controle de inventário). Cada serviço é containerizado, possui health checks configurados e pode ser escalado independentemente.

2. Ambiente Local com Docker Compose

2.1 Arquitetura Multi-serviço

O arquivo docker-compose.yml define todos os serviços necessários para execução local da plataforma. A configuração implementa duas redes isoladas (frontend e backend) para segregação de tráfego, health checks para cada serviço com período de 30 segundos, e dependências com conditions que garantem inicialização ordenada. Todos os containers são executados como usuário não-root (nodejs:1001) e possuem a opção no-new-privileges habilitada.

Funcionalidade	Implementação
Redes isoladas	frontend (172.28.0.0/16) e backend (172.29.0.0/16)
Health checks	Interval: 30s, Timeout: 10s, Retries: 3
Dependências	condition: service_healthy para inicialização ordenada
Segurança	Usuário nodejs:1001, no-new-privileges:true

Tabela 1: Configuração do Docker Compose

2.2 Execução com Único Comando

Todos os serviços são iniciados com o comando ``docker-compose up -d``. As variáveis de ambiente são definidas no arquivo .env (copiado de .env.example), incluindo a API_KEY para autenticação e URLs dos serviços. O README.md contém instruções detalhadas para execução, teste e parada dos serviços, incluindo exemplos de curl para cada endpoint.

3. Containerização e Versionamento

3.1 Dockerfiles Multi-stage

Cada serviço possui um Dockerfile otimizado com multi-stage build. O estágio builder instala as dependências com ``npm ci --only=production``, enquanto o estágio production copia apenas os artefatos necessários. A imagem final é baseada em `node:18-alpine`, resultando em imagens de aproximadamente 150MB. A redução de camadas foi alcançada agrupando comandos RUN e utilizando `npm cache clean --force`.

Prática	Implementação
Multi-stage build	Estágios: builder e production
Imagem base	node:18-alpine (~40MB base)
Usuário não-root	adduser -S nodejs -u 1001
Health check	wget --spider /health a cada 30s
Variáveis de ambiente	NODE_ENV=production, PORT definida

Tabela 2: Melhores práticas de Dockerfile

3.2 Versionamento de Imagens

O pipeline CI/CD gera tags semânticas automáticas (v1.0.X) para releases em main e tags baseadas em SHA para branches de desenvolvimento. A tag 'latest' é atualizada automaticamente. Este esquema permite rollback preciso para qualquer versão anterior e facilita a correlação entre código e artefatos.

4. Kubernetes – Produção Mínima

4.1 Manifests Organizados

Os manifests Kubernetes seguem o padrão Kustomize com estrutura base/overlays. A base contém recursos comuns (namespace, configmaps, secrets, deployments, services, HPA), enquanto overlays definem customizações por ambiente. Cada Deployment configura réplicas, estratégia rolling update (maxSurge/maxUnavailable 25%), probes de liveness e readiness, e limites de recursos (requests: 100m CPU/128Mi mem, limits: 500m CPU/256Mi mem).

4.2 ConfigMaps e Secrets

O ConfigMap loja-veloz-config armazena configurações não sensíveis como URLs dos serviços, ambiente e nível de log. O Secret loja-veloz-secrets contém a API_KEY para autenticação. Nos deployments, são montados via envFrom para injeção automática como variáveis de ambiente. A rotação de secrets pode ser automatizada através do

pipeline.

4.3 Readiness e Liveness Probes

Todos os serviços implementam probes HTTP: liveness em /health (initialDelay: 10s, period: 30s) detecta containers travados e força reinício; readiness em /ready (initialDelay: 5s, period: 10s) garante que apenas Pods prontos recebam tráfego. O API Gateway implementa readiness avançado que verifica disponibilidade de todos os serviços downstream.

4.4 Pod Security Admission

O namespace está configurado com Pod Security Admission nível 'restricted', aplicando as seguintes restrições: runAsNonRoot obrigatório, readOnlyRootFilesystem, allowPrivilegeEscalation false, e drop de todas as capabilities. Esta configuração é aplicada através de labels no namespace e garante conformidade com políticas de segurança.

Configuração	Valor	Justificativa
runAsNonRoot	true	Impede execução como root
runAsUser	1001	UID do usuário nodejs
readOnlyRootFilesystem	true	Previne modificações em runtime
allowPrivilegeEscalation	false	Bloqueia ganho de privilégios
capabilities.drop	ALL	Remove capabilities Linux

Tabela 3: Configuração de segurança dos containers

5. Pipeline CI/CD

O pipeline GitHub Actions (`.github/workflows/ci-cd.yml`) automatiza todo o ciclo de vida da aplicação. Em cada push ou pull request, são executados: lint com ESLint, testes com Jest, scan de vulnerabilidades com Trivy, construção e push de imagens Docker para ghcr.io, e deploy automático para desenvolvimento ou manual para produção.

Estágio	Ferramenta	Validação
Lint	ESLint	Análise estática de código
Test	Jest	Testes unitários e cobertura
Security	Trivy	Vulnerabilidades CRITICAL/HIGH

Build	Docker Buildx	Imagens multi-arch
Deploy Dev	kubectl	Automático após merge
Deploy Prod	kubectl	Requer aprovação manual

Tabela 4: Estágios do pipeline CI/CD

Secrets são gerenciados através do GitHub Secrets (KUBE_CONFIG_DEV, KUBE_CONFIG_PROD), nunca expostos em logs. O deploy em produção exige aprovação manual via ambiente 'production', garantindo controle sobre releases críticos.

6. Observabilidade, Deploy e Escala

6.1 Métricas, Logs e Traces

Todos os serviços implementam logs estruturados em JSON via Winston, com campos: level, message, service, traceId, timestamp. As anotações Prometheus nos Pods permitem scrape de métricas em /metrics. O tracing distribuído é implementado através do header X-Trace-Id, propagado entre serviços e incluído em todas as respostas HTTP.

6.2 Estratégia de Deploy

A estratégia principal é Rolling Update, configurada com maxSurge 25% e maxUnavailable 25%. Esta abordagem garante zero downtime durante atualizações: novos Pods são criados antes de remover os antigos. Para releases de maior risco, pode-se implementar Canary releases (liberar 10% do tráfego inicialmente, aumentar gradualmente) ou Blue-Green (manter duas versões completas, switch instantâneo). A escolha do Rolling Update justifica-se pela simplicidade operacional e eficiência de recursos.

6.3 Escalabilidade (HPA)

O Horizontal Pod Autoscaler está configurado para escalar baseado em CPU (target 70%) e memória (target 80%). O serviço de Pedidos possui min=3 e max=20 réplicas, sendo o mais escalável por ser o ponto central do fluxo de checkout. O comportamento de scaleUp permite dobrar réplicas a cada 15s, enquanto scaleDown tem janela de estabilização de 5 minutos para evitar flapping.

Serviço	Min	Max	CPU Target
API Gateway	2	10	70%
Pedidos	3	20	70%
Pagamentos	2	15	70%

Estoque	2	10	70%
---------	---	----	-----

Tabela 5: Configuração de HPA por serviço

7. Conclusão

Este projeto demonstra a implementação completa de uma plataforma Cloud DevOps seguindo as melhores práticas da indústria. O ambiente local com Docker Compose permite desenvolvimento ágil e reproduzível, enquanto os manifests Kubernetes garantem implantação escalável e segura em produção. O pipeline CI/CD automatiza validações e deploys, reduzindo erros humanos e acelerando entregas.

A arquitetura de microsserviços com observabilidade integrada permite diagnóstico rápido de problemas e escalabilidade independente por serviço. A estratégia Rolling Update combinada com HPA garante alta disponibilidade durante picos de tráfego. As práticas de segurança (Pod Security Admission, containers não-root, secrets management) protegem a plataforma contra vulnerabilidades comuns.

O repositório está pronto para ser clonado e executado com um único comando (`docker-compose up -d`), atendendo aos requisitos de ambiente local funcional e documentação clara. Os manifests Kubernetes podem ser aplicados diretamente via `kubectl apply -k` para implantação em clusters de produção.